Ex. No. 8	MODERN USER INTERFACE DESIGN USING REACTJS
Date of Exercise	13/10/2026
GitHub Host Link	https://github.com/adhishm-0/web-technology-lab/tree/main/EXP8

Objective

To design and develop a modern, responsive user interface using ReactJS, reusable components, form validation, React state, and React Router.

Description

ReactJS is a JavaScript library for building interactive user interfaces from reusable components. This experiment implements a responsive student portal with a validated registration form, client-side navigation, a shared component system, a responsive login page, and a student profile that updates through React state.

IMPORTANT REACT COMMANDS USED :

• useState() — store and update component state

• useMemo() — memoize values derived from state

• onChange — respond to input changes

• onSubmit — handle form submission

• BrowserRouter — enable client-side routing

• Routes and Route — render a page for a URL path

• NavLink — navigate and identify the active route

• map() — render reusable fields and navigation items

Procedure

1. Create the React application and add React Router.

2. Build the shared Navbar, RegistrationForm, and Footer components.

3. Implement field validation and disabled-submit behavior for registration.

4. Add Home, About, and Contact routes with React Router.

5. Create a responsive login page using a desktop and mobile layout.

6. Use React state to update student details and the live profile card.

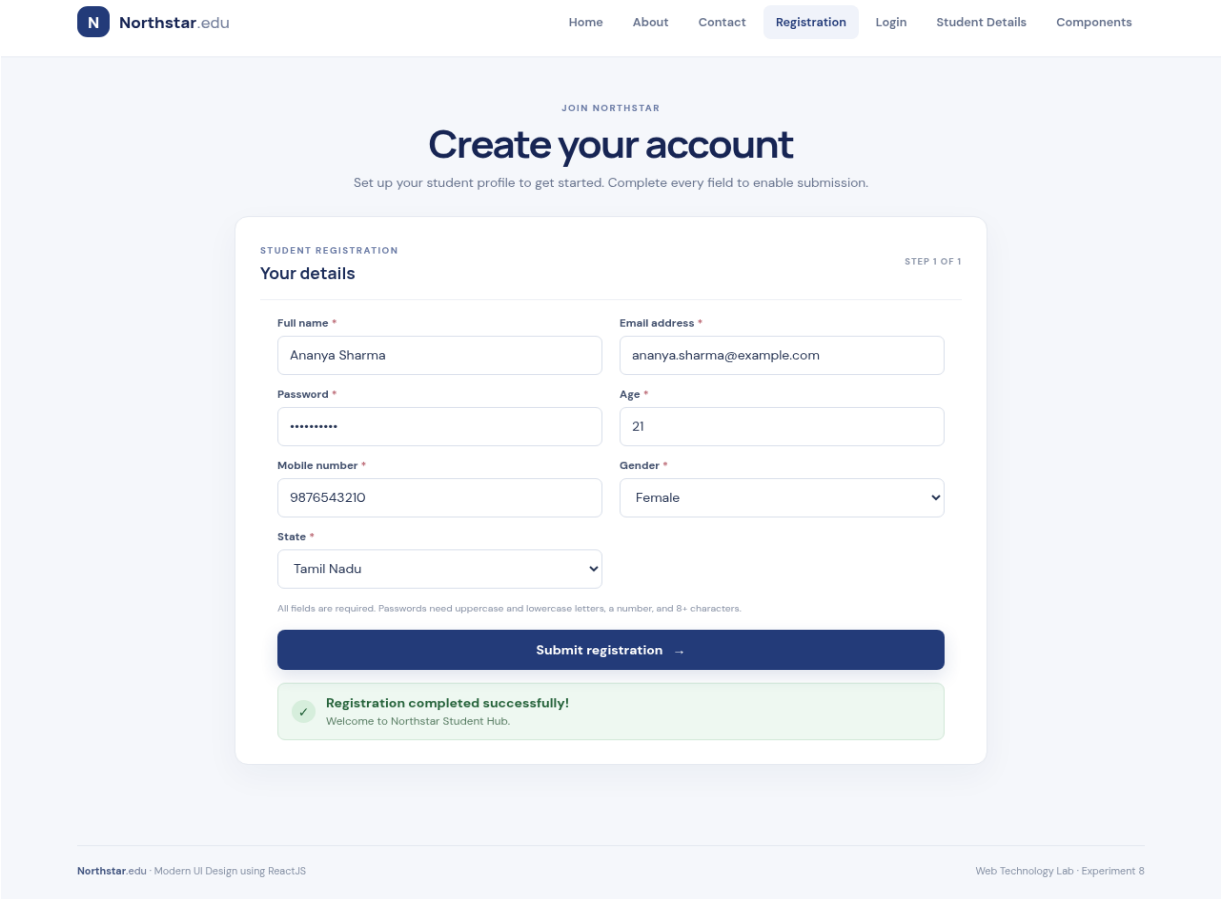
7. Run the application and capture the output for all five questions.

Question 1: Responsive Registration Form

Program code:

```
function validate(values) {
  const errors = {};
  if (!/^[A-Za-z ]+$/.test(values.name.trim())) errors.name = 'Use letters and spaces only.';
  if (!/(?=.*[a-z])(?=.*[A-Z])(?=.*\d).{8,}/.test(values.password)) errors.password = 'Use 8+
  characters with uppercase, lowercase and a number.';
  if (Number(values.age) < 18 || Number(values.age) > 30) errors.age = 'Age must be between 18
  and 30.';
  if (!/^\d{10}$/.test(values.mobile)) errors.mobile = 'Enter exactly 10 digits.';
  if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(values.email)) errors.email = 'Enter a valid email.';
  if (!values.gender) errors.gender = 'Select a gender.';
  if (!values.state) errors.state = 'Select a state.';
  return errors;
}
const errors = validate(values);
const isValid = Object.keys(errors).length === 0;
<button type="submit" disabled={!isValid}>Submit registration</button>
{submitted && <div role="status">Registration completed successfully!</div>}
```

Program Output:



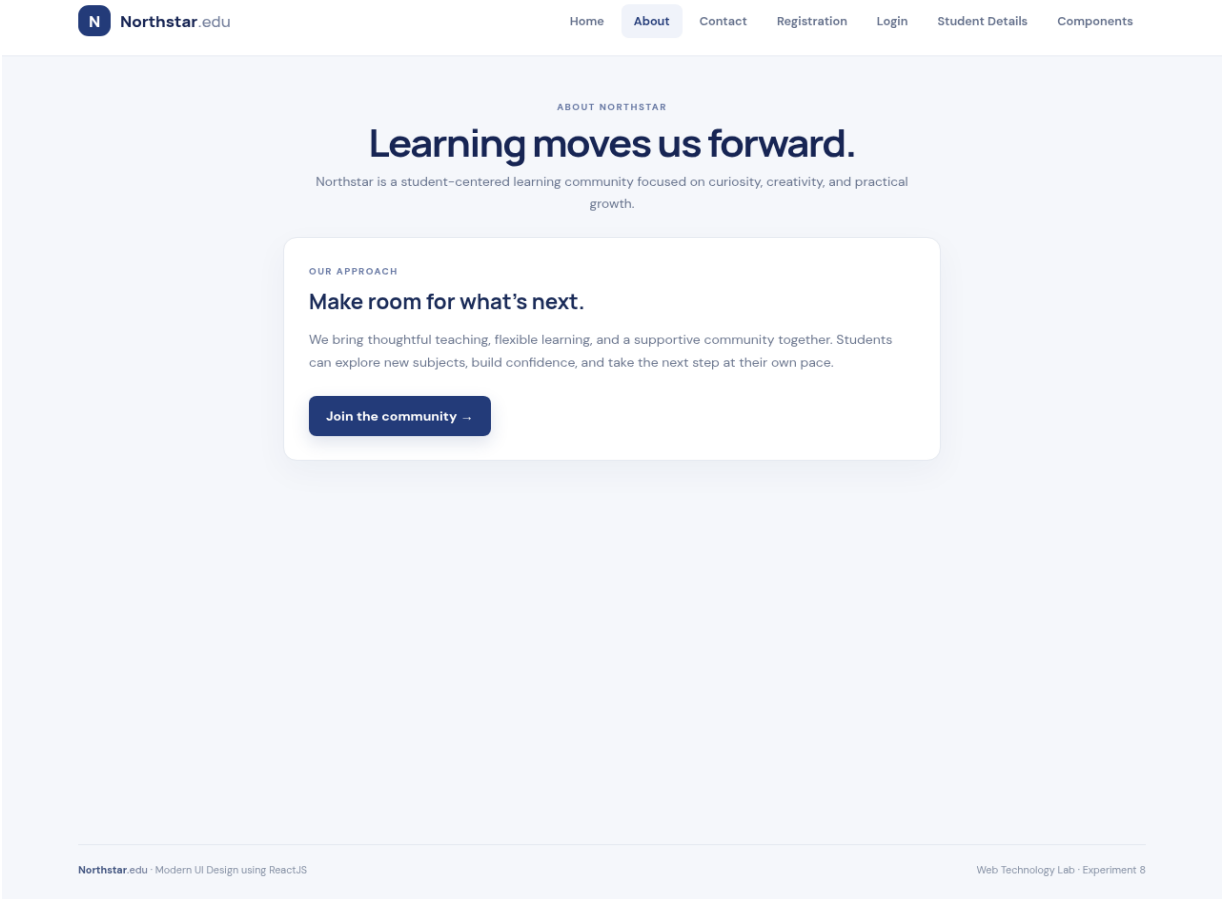
Output 1: The responsive React interface demonstrates responsive registration form functionality.

Question 2: React Router Navigation

Program code:

```
import { BrowserRouter, Routes, Route, NavLink } from 'react-router-dom';
function Navbar() {
  return <nav aria-label="Main navigation">
    <NavLink to="/">Home</NavLink>
    <NavLink to="/about">About</NavLink>
    <NavLink to="/contact">Contact</NavLink>
  </nav>;
}
function App() {
  return <BrowserRouter>
    <Navbar />
    <Routes>
      <Route path="/" element={<Home />} />
      <Route path="/about" element={<About />} />
      <Route path="/contact" element={<Contact />} />
    </Routes>
  </BrowserRouter>;
}
```

Program Output:



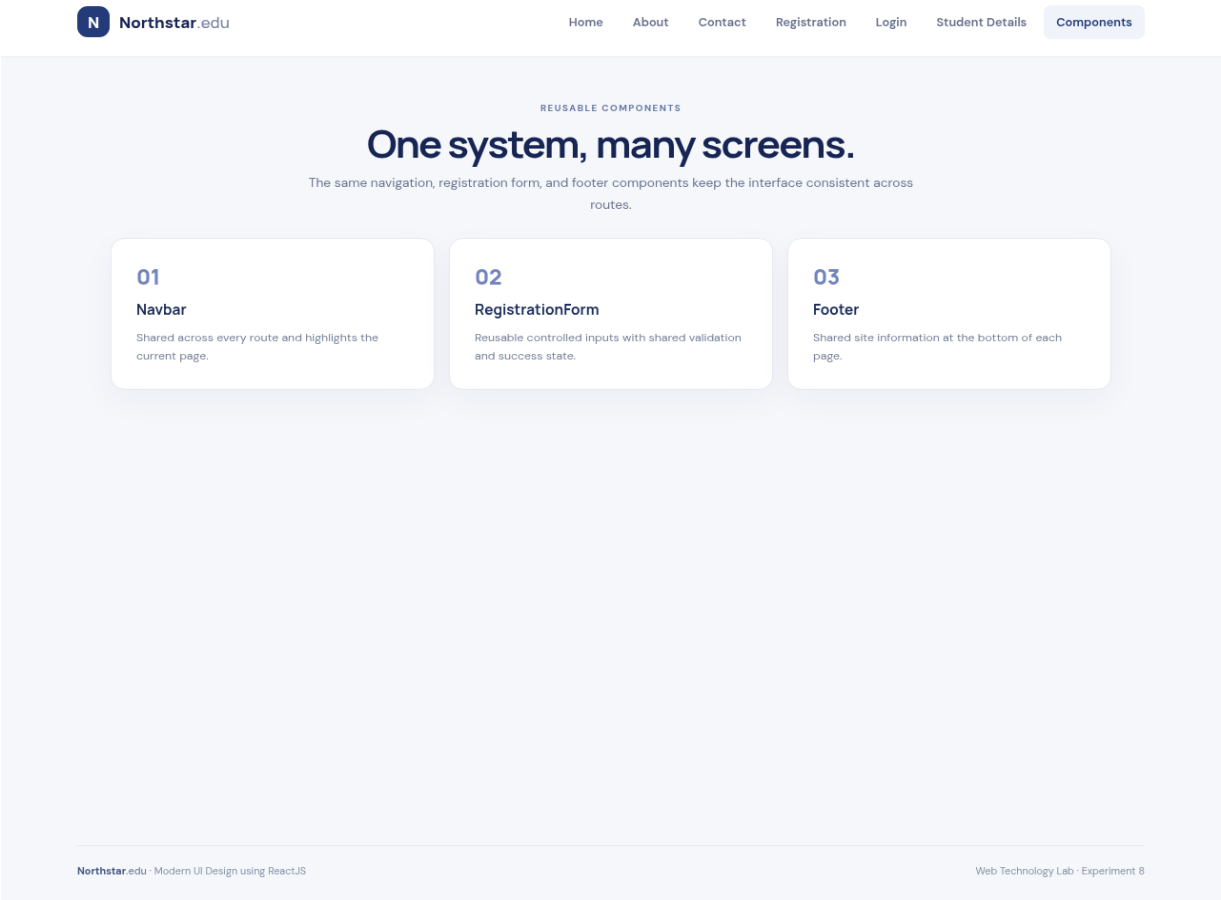
Output 2: The responsive React interface demonstrates react router navigation functionality.

Question 3: Reusable React Components

Program code:

```
function Navbar() {
  return <header className="site-header"><nav className="main-nav">
    {links.map(([path, label]) => <NavLink key={path} to={path}>{label}</NavLink>)}
  </nav></header>;
}
function RegistrationForm({ onSubmit }) {
  const [values, setValues] = useState(initialValues);
  const errors = validate(values);
  const valid = Object.keys(errors).length === 0;
  return <form onSubmit={onSubmit}>
    {fields.map(field => <FormField key={field.name} {...field} value={values[field.name]} />)}
    <button disabled={!valid}>Submit registration</button>
  </form>;
}
function Footer() {
  return <footer className="site-footer">Northstar.edu · Modern UI Design using
  ReactJS</footer>;
}
// Shared layout renders the same components on every route.
<><Navbar /><main><Routes>{/* page routes */}</Routes></main><Footer /></>
```

Program Output:



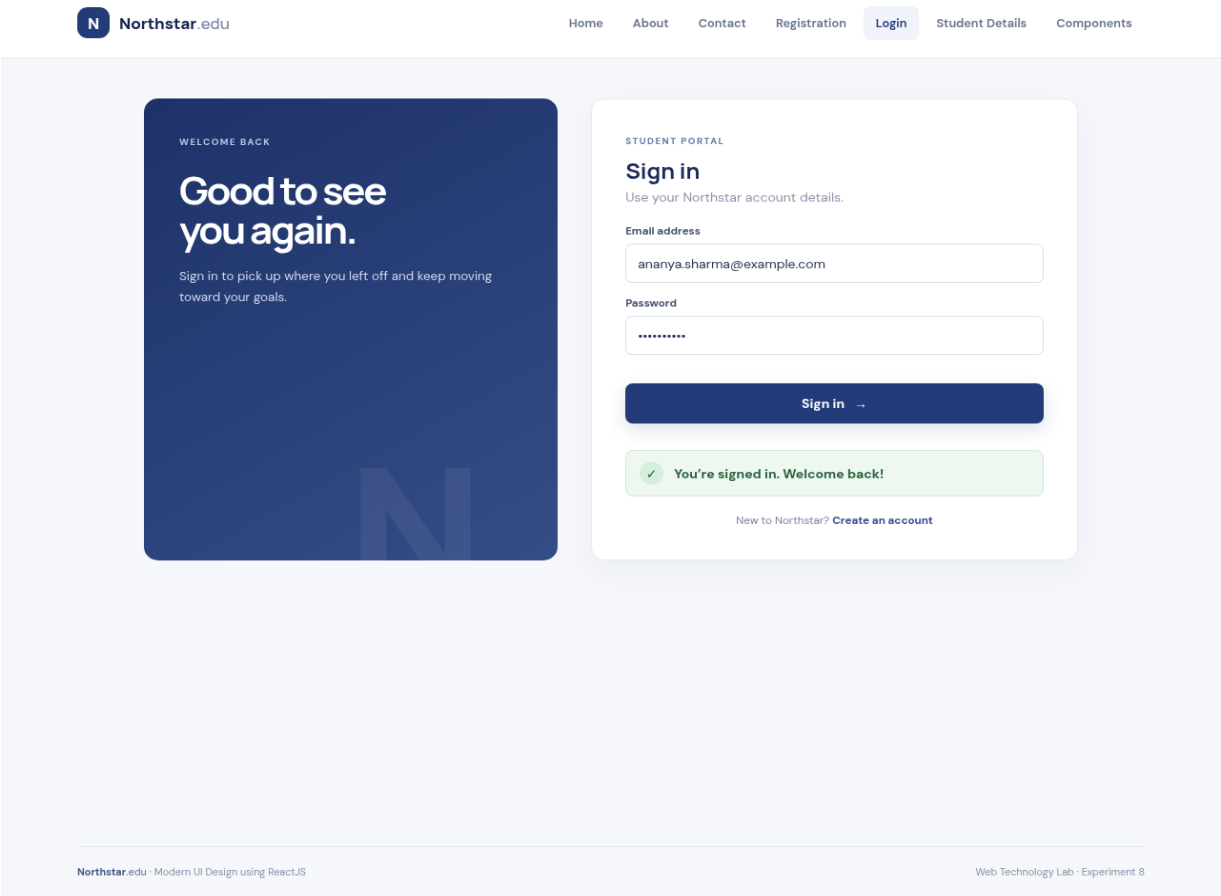
Output 3: The responsive React interface demonstrates reusable react components functionality.

Question 4: Responsive Login Page

Program code:

```
function LoginPage() {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [message, setMessage] = useState('');
  function login(event) {
    event.preventDefault();
    setMessage(email && password
      ? 'You are signed in. Welcome back!'
      : 'Enter your email and password to continue.');
  }
  return <div className="login-layout">
    <aside className="login-aside"><h1>Good to see you again.</h1></aside>
    <section className="content-card login-card">
      <form onSubmit={login}>
        <input type="email" value={email} onChange={e => setEmail(e.target.value)} required />
        <input type="password" value={password} onChange={e => setPassword(e.target.value)}
  required />
        <button className="button button-primary">Sign in</button>
        {message && <div role="status">{message}</div>}
      </form>
    </section>
  </div>;
}
/* CSS media queries reflow the two-column layout for tablets and phones. */
```

Program Output:



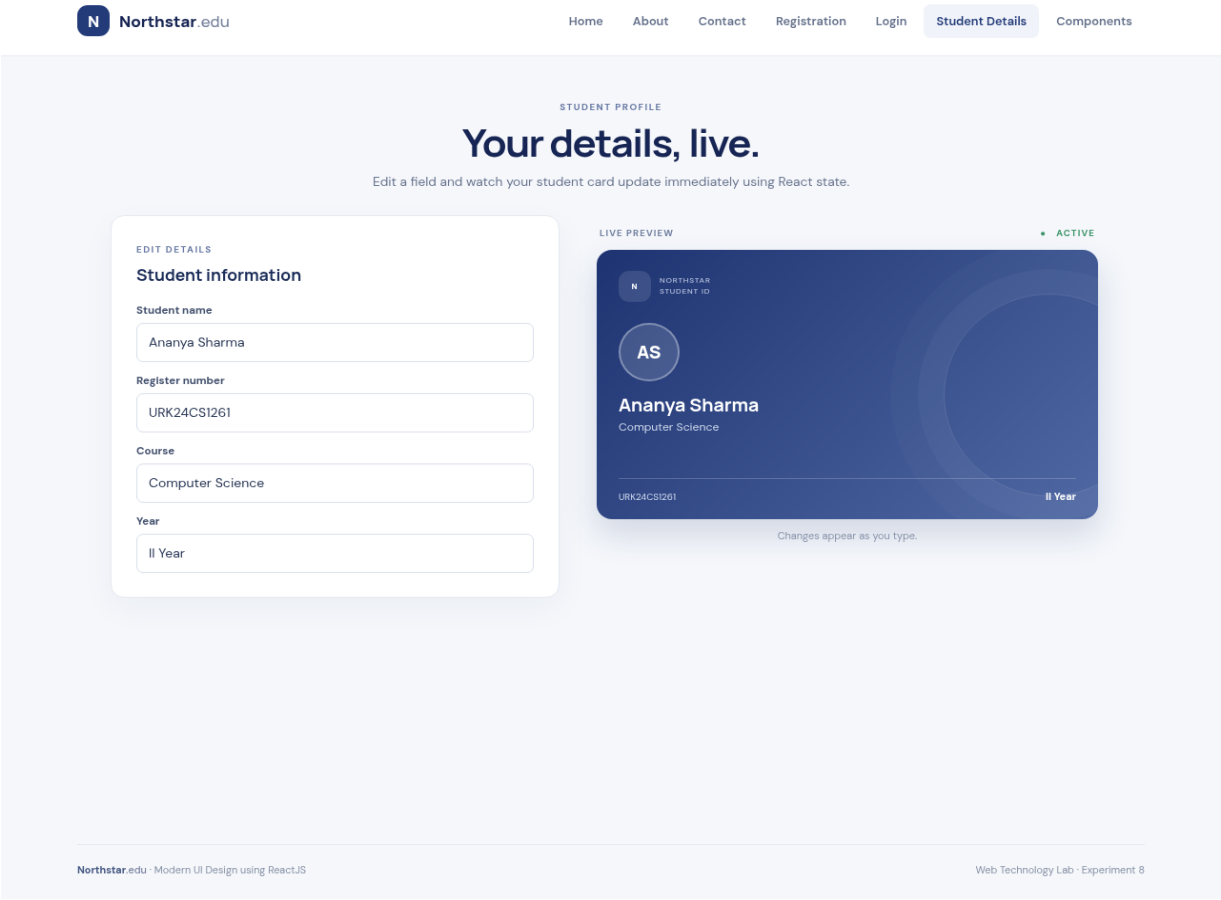
Output 4: The responsive React interface demonstrates responsive login page functionality.

Question 5: Dynamic Student Details with State

Program code:

```
function StudentPage() {
  const [student, setStudent] = useState({
    name: 'Ananya Sharma', register: 'URK24CS1261',
    course: 'Computer Science', year: 'II Year'
  });
  function update(event) {
    setStudent(current => ({ ...current, [event.target.name]: event.target.value }));
  }
  return <div className="student-layout">
    <section className="student-editor">
      <label>Student name<input name="name" value={student.name} onChange={update} /></label>
      <label>Register number<input name="register" value={student.register} onChange={update}
  /></label>
      <label>Course<input name="course" value={student.course} onChange={update} /></label>
      <label>Year<input name="year" value={student.year} onChange={update} /></label>
    </section>
    <article className="student-card">
      <h2>{student.name}</h2><p>{student.course}</p>
      <span>{student.register}</span><strong>{student.year}</strong>
    </article>
  </div>;
}
```

Program Output:



Output 5: The responsive React interface demonstrates dynamic student details with state functionality.

Result

The modern ReactJS user interface was designed and implemented successfully. The application demonstrates responsive page layouts, reusable components, validated registration, route-based navigation, interactive login, and student details that update with React state.